Le moteur de template Jinja

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Les moteurs de templates **3**

A. Principe de fonctionnement 3

B. Les différents moteurs de template 5

C. Exercice : Quiz 6

III. Utiliser le moteur de template Jinja2 **7**

A. Les templates de base 7

B. Insérer des instructions 8

C. Héritage 9

D. Exercice : Quiz 11

IV. Essentiel **12**

V. Auto-évaluation **12**

A. Exercice 12

B. Test 13

Solutions des exercices **13**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+, PyCharm Community

Pré requis : bases en Python

Contexte

Par définition, une application Web Python est susceptible de fournir du contenu dynamique. *A contrario*, les pages html affichent du contenu statique. Comment alors peut-on générer un contenu statique, au format html, à partir de données dynamiques, qui sont déterminées par les différentes routines implémentées dans notre application Web ?

La réponse réside dans l'utilisation de ce qu'on appelle des moteurs de template. Dans ce cours, nous verrons de quoi il s'agit, comment ils fonctionnent et quels sont les différents moteurs de template disponibles.

Nous montrerons avec quelques exemples simples comment ils peuvent être utilisés efficacement.

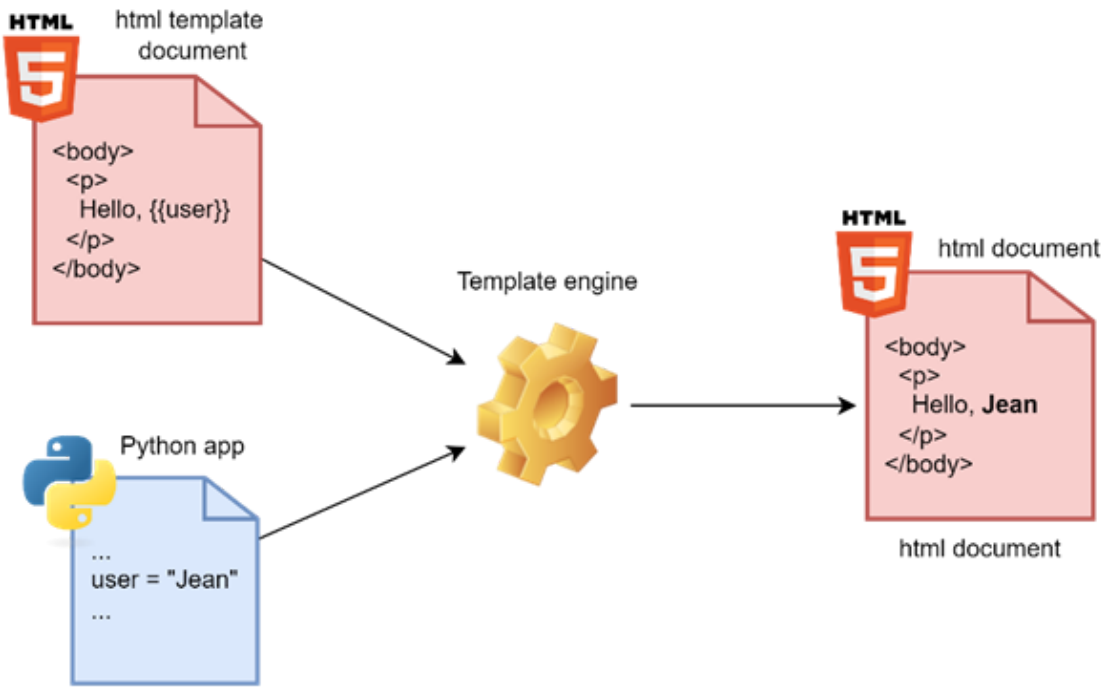
II. Les moteurs de templates

A. Principe de fonctionnement

Par nature, les documents html sont destinés à afficher du contenu statique. Cependant, cette contrainte risque de rapidement devenir un problème de taille.

Imaginez par exemple que vous souhaitez, suite à l'authentification d'un utilisateur, afficher simplement « *Bonjour, <utilisateur>* » à l'écran. La variable <utilisateur> doit évidemment s'adapter à l'utilisateur qui vient de s'authentifier : cela doit être un contenu « *dynamique* ».

Il est donc nécessaire de générer « *à la volée* » un contenu html statique, mais dans lequel on a au préalable « *injecté* » le contenu d'une variable « *utilisateur* ». C'est exactement ce à quoi servent les moteurs de templates, appelés en anglais « *Template engines* ».



Les moteurs de template peuvent fonctionner de deux manières différentes :

- Soit ils sont exécutés coté client.
- Soit ils sont exécutés coté serveur.

Source : HACKMANIT[1]

Dans le premier cas, voici comment cela fonctionne :

1. Le client envoie une requête http (requête GET) vers le serveur afin d'obtenir le contenu d'une page web,
2. Le serveur répond en fournissant deux éléments :
    1. Le template html,
    2. Le script qui doit être utilisé pour interpréter ce template

Cependant, ce type de fonctionnement n'est possible que si le client est capable d'exécuter du code (puisque le client doit interpréter le script spécifié par le serveur).

Actuellement, le seul type de script qui peut être exécuté nativement par un navigateur web est le JavaScript. Il n'est actuellement pas possible d'interpréter du code Python nativement dans un navigateur Web.

Remarque

Le JavaScript est un langage de programmation qui, comme Python, est interprété (et non compilé en langage machine). Il est actuellement le seul langage de programmation qui soit supporté nativement par tous les navigateurs Web : Chrome, Edge, Firefox, etc.

Dans le deuxième cas, le moteur de template est situé coté serveur (c'est toujours le cas pour les applications Web développées en Python, par exemple avec Flask).

Dans ce scénario, voici comment cela fonctionne :

1. Le client envoie une requête http au serveur
2. Le serveur génère une page web html statique sur base de son application Python et de son template.

1. https://www.hackmanit.de/

Tous droits réservés à STUDI - Reproduction interdite

Remarque **Attaque par injection coté serveur**

Nous venons de voir qu'il est possible d'envoyer du contenu vers le serveur, qui peut être interprété par ce dernier pour renvoyer un contenu html dynamique.

Lorsqu'on s'authentifie, le serveur interprète les éléments envoyés (par exemple le nom d'utilisateur), pour pouvoir les injecter dans un template html. Ceci signifie que si nous envoyons par exemple « *2+3* » au lieu de « *Jean* », le serveur devrait nous renvoyer « *Bonjour, 5* » dans le premier cas, et « *Bonjour, Jean* » dans le second.

Ceci signifie qu'il est en théorie possible de faire interpréter n'importe quoi par le serveur : remplacer « *2+3* » par une commande système, et vous seriez capable de faire exécuter un peu près n'importe quelle commande sur le serveur distant. C'est ce qu'on appelle communément une « *attaque par injection* ».

Pour remédier à ce problème, il est important d'appliquer les précautions suivantes :

1. **Limiter le « *scope* » de ce qu'il est possible d'atteindre à travers les templates html** (généralement, la configuration du moteur de template permet de réaliser cela facilement)
2. **Essayez toujours de contraindre les champs qui sont éditables par l'utilisateur**. Par exemple, n'acceptez pas de chiffres dans un champ censé prendre uniquement des lettres, et inversement, ou bien limitez le nombre de caractères possibles
3. **« *Echappez* » les contenus utilisateur**. « *Echapper* » est une traduction littérale de « *Escape* », qui signifie plutôt « *Passiver* ». Ainsi, le contenu lui-même ne sera plus interprété par le script coté serveur, il sera considéré comme « *statique* ».

Par ailleurs, le moteur Jinja est capable d'interpréter les entrées dans une « *sandbox* », c'est-à-dire littéralement un « *bac à sable* ». Ceci consiste à créer un environnement local et spécifique, qui n'a accès qu'à un ensemble très restreint de ressources locales.

B. Les différents moteurs de template

Il existe plusieurs moteurs de template disponibles. En Python, on citera par exemple :

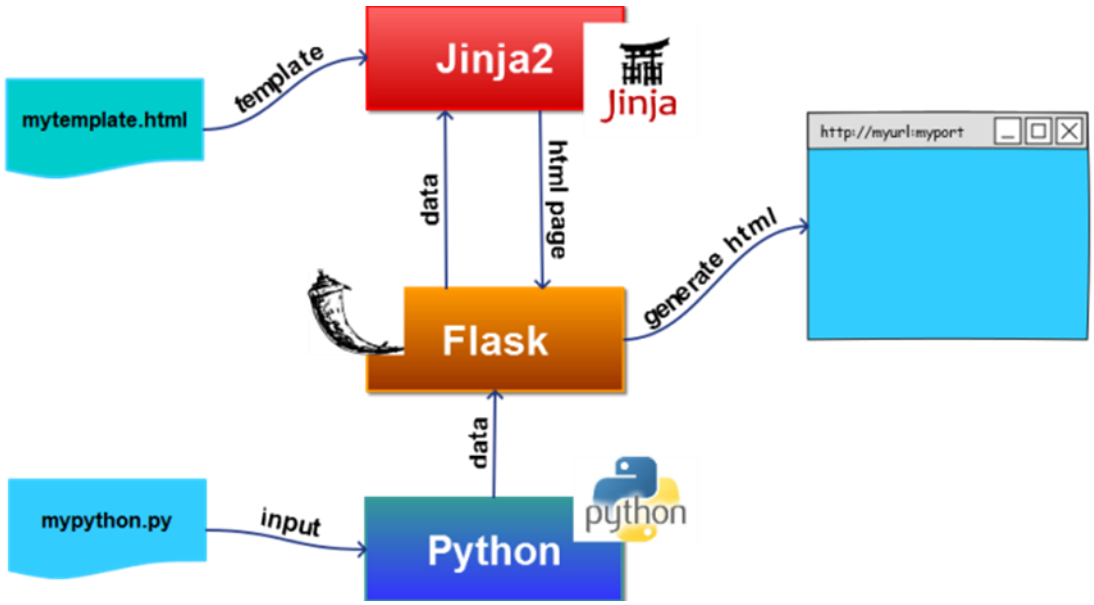
- Jinja (et Jinja 2)
- Django templates
- Mako

Il en existe d'autres, mais ce sont les principaux et les plus employés.

Dans ce cours, nous utiliserons Jinja 2. C'est un des moteurs de template les plus employés, d'autant plus avec le framework Flask, que nous avons déjà eu l'occasion d'utiliser (et que nous avons déjà utilisé sans le savoir !). Flask intègre d'ailleurs nativement le moteur de template Jinja2.

Tous droits réservés à STUDI - Reproduction interdite

Voici à titre d'illustration, un diagramme qui illustre les différentes interactions entre Flask et Jinja pour « *servir* » un contenu html à partir d'une applications Python et d'un template html :



Source : gertjanvanhethofblog[2]

Remarque

Le nom « *Jinja* » vient du fait que « *Jinja* » signifie « *Temple* » en japonais. « *Temple* » et « *Template* » ont les mêmes prononciations, c'est la raison pour laquelle ce moteur de template s'appelle « *Jinja* ». « *Jinja* » est également le nom d'une ville d'Ouganda, mais l'origine du nom de ce moteur de template ne provient pas de là.

C. Exercice : Quiz

[solution n°1 p.15]

Question 1

Pourquoi faut-il utiliser un moteur de template ?

- Pour accélérer le chargement de son site web
- Pour pouvoir interpréter du code Python
- Pour pouvoir rendre des pages html dynamiques

Question 2

Jinja est l'unique moteur de template Python.

- Vrai
- Faux

Question 3

Flask intègre nativement Jinja2 :

- Vrai
- Faux

2. https://gertjanvanhethofblog.wordpress.com/

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Qu'est-ce qu'une attaque par injection côté serveur ?

- Un risque quasiment inexistant car les navigateurs sont protégés
- Un risque inexistant si https (http sécurisé) est utilisé
- Un risque réel qu'une personne malveillante exécute des commandes côté serveur

Question 5

Il est possible d'utiliser un moteur de template coté client en Python.

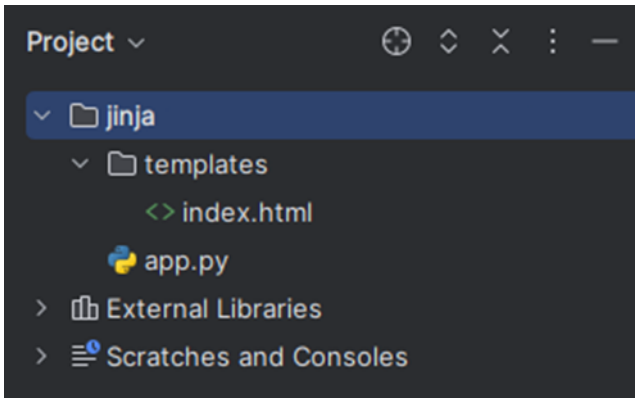
- Vrai
- Faux

III. Utiliser le moteur de template Jinja2

A. Les templates de base

Si vous avez déjà installé la librairie Flask, vous n'avez pas besoin d'installer spécifiquement Jinja2, celui-ci étant livré nativement avec Flask.

Commençons par créer un projet qui a l'organisation suivante :



Le dossier « *template* » contient un fichier index.html. Notre application est représentée par le fichier app.py.

Dans notre fichier index.html, nous allons écrire le contenu suivant :

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>{{ pageTitle }}</title>
6 </head>
7 <body>
8 <p>{{ content }}</p>
9 </body>
10 </html>
```

Comme nous pouvons le constater, il s'agit bien d'un template : des éléments dynamiques sont notés entre crochets {{ … }}.

Cette notation permet au moteur de template de savoir qu'il faut remplacer ce contenu par les variables portant le même nom dans notre application.

Dans ce cas, nous avons défini deux éléments dynamiques : le titre « *pageTitle* » et le contenu de notre paragraphe « *content* ».

Tous droits réservés à STUDI - Reproduction interdite

Passons maintenant au fichier app.py, voici son contenu :

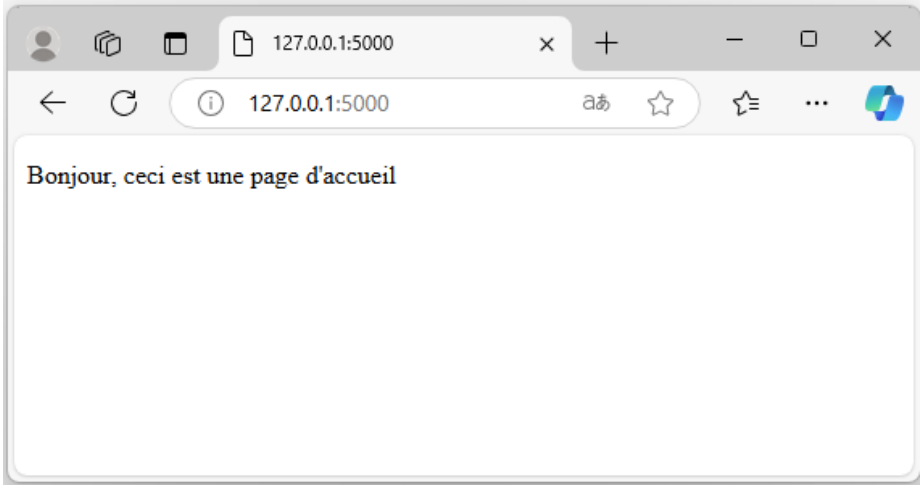
```
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5
6 @app.route('/')
7 def home():
8     return render_template('index.html', title='Accueil', content="Bonjour, ceci est une page d'accueil")
```

Remarquez l'utilisation de la fonction « *render_template()* ». On a ici affaire à une fonction fournie par la librairie Jinja : il s'agit de « *rendre* » un template (ici, index.html), en spécifiant les éléments « *dynamiques* » de ce template.

Comme toujours pour exécuter et tester notre application, exécutez la commande suivante :

```
> flask run
```

Et rendez-vous sur le serveur de développement : http://127.0.0.1:5000[3] , vous devriez voir le contenu suivant :



B. Insérer des instructions

Il est également possible d'insérer des instructions directement dans la page html. Ceci se fait en utilisant la notation suivante :

```
1 {% instruction %}
```

Voyons ensemble un exemple.

Exemple

Supposons que nous disposions d'une liste d'utilisateurs dans notre application, et que nous voulons afficher cette liste dans notre page html.

Pour ce faire, commençons par définir la liste d'utilisateurs :

```
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5 myUsers = ["Jean", "Arnaud", "Michael"]
6
```

3. http://127.0.0.1:5000/

Tous droits réservés à STUDI - Reproduction interdite

```
7 @app.route('/')
8 def home():
9     return render_template('index.html', pageTitle='Accueil', content="Bonjour, ceci est une page d'accueil", users=myUsers)
```

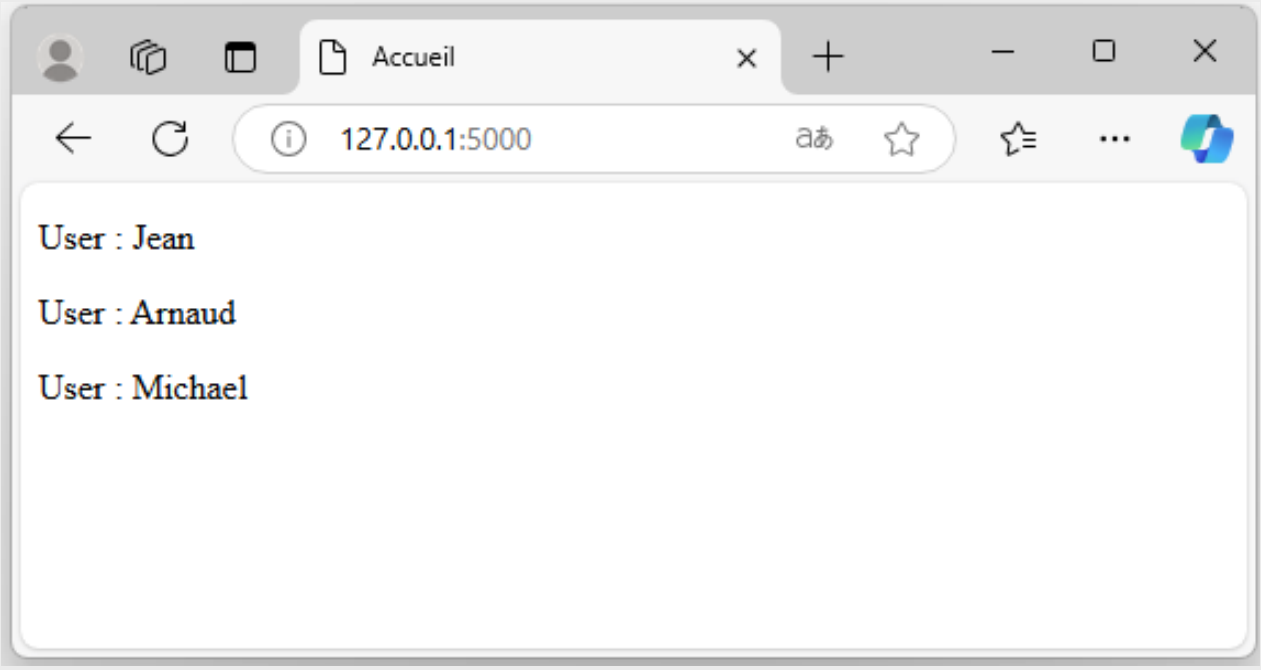
Notez que la fonction render_template() assigne désormais « *users* = *myUsers* ». On passe donc la liste myUsers au moteur de template Jinja.

Côté html, nous avons :

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>{{ pageTitle }}</title>
6 </head>
7 <body>
8     {% for user in users: %}
9     <p> User : {{ user }}</p>
10    {% endfor %}
11 </body>
12 </html>
```

Notez ici l'utilisation de l'instruction {% endfor %}, qui permet de préciser la fin de notre boucle for.

Une fois votre serveur relancé, vous devriez obtenir le contenu suivant :



C. Héritage

Quand vous développez une application Web, vous serez très souvent amené à utiliser une page de base, et à l'étendre avec du contenu différent d'une page à l'autre. Ceci peut être réalisé grâce à l'héritage. Voyons ensemble un exemple concret.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Commençons par légèrement modifier notre page index.html :

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>{{ pageTitle }}</title>
6 </head>
7 <body>
8     {% for user in users: %}
9     <p> User : {{ user }}</p>
10    {% endfor %}
11
12    {% block content %}
13    {% endblock %}
14 </body>
15 </html>
```

Remarquez l'utilisation de l'instruction {% block content %} … {% endblock %} qui permet de définir où, dans notre page index.html, sera située l'extension.

Créons un second fichier .html que nous appellerons extend.html :

```
1 {% extends "index.html" %}
2 {% block content %}
3     <p> Nombre total d'utilisateurs : {{ users|count }} </p>
4 {% endblock %}
```

Notez que puisque ce fichier hérite de index.html, il n'est pas nécessaire de ré-écrire l'ensemble des tags d'une page html. Ce sont ceux de la page parente qui seront ré-utilisés.

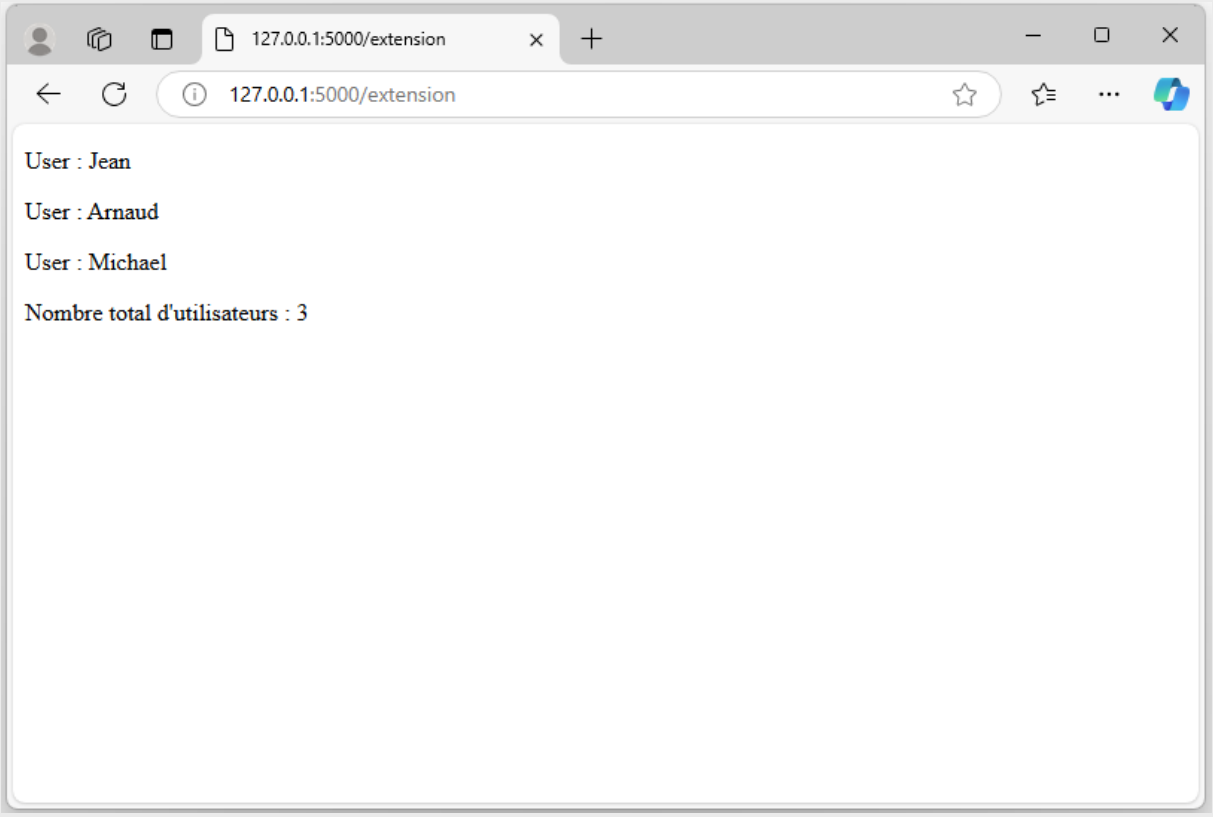
Il nous reste à définir une route dans notre application Python qui pointera vers cette page extend.html :

```
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5 myUsers = ["Jean", "Arnaud", "Michael"]
6
7 @app.route('/')
8 def home():
9     return render_template('index.html', pageTitle='Accueil', content="Bonjour, ceci est une
  page d'accueil", users=myUsers)
10
11 @app.route('/extension')
12 def extend():
13     return render_template('extend.html', users=myUsers)
```

Comme on peut le voir, cette page sera récupérée en accédant à l'adresse « */extension* ».

Tous droits réservés à STUDI - Reproduction interdite

Si vous accédez maintenant à cette page, via l'adresse 127.0.0.1:5000/extension, vous devriez obtenir le résultat suivant :



On peut voir que notre page extend.html « *étend* » effectivement notre page de base index.html.

D. Exercice : Quiz

[solution n°2 p.16]

Exercice

Comment afficher le contenu d'une variable avec Jinja ?

- [myVariable]
- "myVariable"
- {{ myVariable }}

Question 2

Comment exécuter une instruction avec Jinja ?

- [& myInstruction &]
- run(myInstruction)
- {% myInstruction %}

Question 3

Tous droits réservés à STUDI - Reproduction interdite

La fonction render_template() permet de ...

- De rendre un template complété par des informations dynamiques obtenues depuis l'application
- De renvoyer vers une url spécifique externe
- Cette fonction n'est pas définie dans Jinja

Question 4

Une extension (héritage) permet d'hériter de toutes nos variables applicatives dans une page web ?

- Vrai
- Faux

Question 5

Comment écrire une boucle for avec Jinja ?

- { for user in users } … { endfor }
- {% for user in users %} … {% endfor %}
- { for user in users } …

IV. Essentiel

Un moteur de template permet de générer du contenu html dynamique à partir de templates et d'une application exécutée coté serveur.

Jinja est un moteur de template (template engine) extrêmement répandu, surtout lorsqu'on souhaite développer une application Web à l'aide du framework Flask. Bien que ce soit un des plus connus, il en existe d'autres, comme par exemple Django templates ou encore Mako.

Nous avons vu comment Jinja pouvait nous aider à générer du contenu dynamique, en affichant par exemple le contenu de nos variables, ou bien en réalisant directement des instructions Python dans une page Web. Nous avons également vu les notions d'héritage, qui permettent à une page html d'hériter directement d'une autre page.

Lors de l'utilisation de moteurs de template, il est important de se prémunir des attaques par injection, qui pourraient compromettre l'intégrité et la sécurité du serveur sur lequel est hébergé notre site web.

V. Auto-évaluation

A. Exercice

Supposons que vous disposiez d'une liste d'utilisateurs, et d'une liste d'adresses mail correspondant à ces utilisateurs :

users = [« *user1* », « *user2* », « *user3* »]

emails = [« *user1@domain.com* », « *user2@domain.com* », « *user3@domain.com* »]

Question 1

[solution n°3 p.17]

Écrivez une application Web qui permet, grâce à Jinja, de lister l'ensemble des utilisateurs et de leur adresse mail sur une même page html.

Question 2

[solution n°4 p.17]

Écrivez une page html qui hérite de la première, et qui permet d'afficher, en plus, le nombre total d'utilisateurs dans la liste.

Tous droits réservés à STUDI - Reproduction interdite

B. Test

Exercice 1 : Quiz

[solution n°5 p.18]

Question 1

Un moteur de template est indispensable pour une application Web.

- Vrai
- Faux

Question 2

Jinja trouve l'origine de son nom dans un temple japonais.

- Vrai
- Faux

Question 3

Comment afficher le nombre d'éléments dans une liste avec Jinja ?

- {{ len(list) }}
- {{ list.size() }}
- {{ list | count }}

Question 4

Il est possible d'utiliser Jinja sans utiliser Flask.

- Vrai
- Faux

Question 5

L'utilisation d'un moteur de template augmente le risque en termes de cybersécurité.

- Vrai
- Faux

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 6 Solution n°1

Question 1

Pourquoi faut-il utiliser un moteur de template ?

- ○ Pour accélérer le chargement de son site web
- ○ Pour pouvoir interpréter du code Python
- ◉ Pour pouvoir rendre des pages html dynamiques

Un moteur de template permet de rendre des pages html dynamiques, en y injectant un contenu généré à la volée.

Question 2

Jinja est l'unique moteur de template Python.

- ○ Vrai
- ◉ Faux

C'est Faux, il existe de nombreux autres moteurs de template, comme par exemple Django templates.

Question 3

Flask intègre nativement Jinja2 :

- ◉ Vrai
- ○ Faux

Flask intègre effectivement le moteur de template Jinja2. Il n'est pas nécessaire d'installer Jinja2 si Flask est déjà présent sur votre machine.

Question 4

Qu'est-ce qu'une attaque par injection côté serveur ?

- ○ Un risque quasiment inexistant car les navigateurs sont protégés
- ○ Un risque inexistant si https (http sécurisé) est utilisé
- ◉ Un risque réel qu'une personne malveillante exécute des commandes côté serveur

Une attaque par injection permet à une personne malveillante d'exécuter du code sur la machine distante grâce aux templates html. Il faut donc prendre des mesures adéquates afin de s'en prémunir.

Question 5

Il est possible d'utiliser un moteur de template coté client en Python.

- ○ Vrai
- ◉ Faux

Les navigateurs web sont incapables d'interpréter du langage Python directement. Par conséquent, les moteurs de template Python sont situés côté serveur.

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 11 Solution n°2

Exercice

Comment afficher le contenu d'une variable avec Jinja ?

- ○ [myVariable]
- ○ "myVariable"
- ◉ {{ myVariable }}

Pour afficher le contenu d'une variable, il faut utiliser l'instruction {{ myVariable }}.

Question 2

Comment exécuter une instruction avec Jinja ?

- ○ [& myInstruction &]
- ○ run(myInstruction)
- ◉ {% myInstruction %}

Une instruction s'utilise avec la notation {% instruction %}. Par exemple : {% for row in rows %}…{% endfor %}.

Question 3

La fonction render_template() permet de ...

- ◉ De rendre un template complété par des informations dynamiques obtenues depuis l'application
- ○ De renvoyer vers une url spécifique externe
- ○ Cette fonction n'est pas définie dans Jinja

Cette fonction permet de récupérer un template, de le compléter avec le contenu de l'application Python, et de le renvoyer vers l'utilisateur.

Question 4

Une extension (héritage) permet d'hériter de toutes nos variables applicatives dans une page web ?

- ○ Vrai
- ◉ Faux

Un héritage permet à une page html d'hériter d'une autre page html. Ceci permet par exemple de réaliser plusieurs pages avec un style identique à partir d'un même template html.

Question 5

Comment écrire une boucle for avec Jinja ?

- ○ { for user in users } … { endfor }
- ◉ {% for user in users %} … {% endfor %}
- ○ { for user in users } …

Tous droits réservés à STUDI - Reproduction interdite

La bonne notation est : {% for user in users %} … {% endfor %}.

p. 12 Solution n°3

Le code suivant répond à la question :

```
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5 myUsers = ["user1", "user2", "user3"]
6 myEmails = ["user1@domain.com", "user2@domain.com", "user3@domain.com"]
7
8 @app.route('/')
9 def home():
10     return render_template('index.html', pageTitle='Accueil', data=list(zip(myUsers,
myEmails)))
```

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>{{ pageTitle }}</title>
6 </head>
7 <body>
8     {% for user, email in data: %}
9     <p> User : {{ user }}, Email : {{email}}</p>
10    {% endfor %}
11
12    {% block content %}
13    {% endblock %}
14 </body>
15 </html>
```

On voit que ce code ressemble fort aux exemples réalisés durant le cours. Les modifications principales concernent l'ajout d'un 2eme tableau, contenant les adresses mails des utilisateurs, ainsi que l'utilisation de la fonction zip() de Python, qui permet de générer un itérateur basé sur des tuples (ici :l'utilisateur et son adresse mail).

La page index.html est modifiée de sorte que la boucle for utilise désormais deux éléments plutôt qu'un seul : le nom d'utilisateur et son adresse mail, et les affiche à l'écran.

p. 12 Solution n°4

La page extends.html contiendra le code suivant :

```
1 {% extends "index.html" %}
2 {% block content %}
3     <p> Nombre total d'utilisateurs : {{ data|count }}</p>
4 {% endblock %}
```

On voit qu'on affiche tout simplement le résultat du filtre « *count* » appliqué à la donnée « *data* ».

Tous droits réservés à STUDI - Reproduction interdite

L'application (notre fichier app.py) doit être modifiée pour y ajouter une nouvelle route, qui pointera vers « *extends.html* » :

```
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5 myUsers = ["user1", "user2", "user3"]
6 myEmails = ["user1@domain.com", "user2@domain.com", "user3@domain.com"]
7
8 @app.route('/')
9 def home():
10     return render_template('index.html', pageTitle='Accueil', data=list(zip(myUsers,
  myEmails)))
11
12 @app.route('/extension')
13 def extend():
14     return render_template('extend.html', data=list(zip(myUsers, myEmails)))
```

Exercice p. 13 Solution n°5

Question 1

Un moteur de template est indispensable pour une application Web.

- (●) Vrai
- ( ) Faux

Pour faire une application Web, il est inévitable de recourir à l'utilisation d'un moteur de template.

Question 2

Jinja trouve l'origine de son nom dans un temple japonais.

- (●) Vrai
- ( ) Faux

Vrai, Jinja signifie « *Temple* » en Japonais, ce qui est, d'un point de vue prononciation, très proche de « *Template* ».

Question 3

Comment afficher le nombre d'éléments dans une liste avec Jinja ?

- ( ) {{ len(list) }}
- ( ) {{ list.size() }}
- (●) {{ list | count }}

Pour afficher le nombre d'éléments dans une liste, il faut faire appel à un « *filtre* » Jinja. En l'occurrence dans ce cas-ci, il s'agit du filtre « *count* ».

Question 4

Il est possible d'utiliser Jinja sans utiliser Flask.

Tous droits réservés à STUDI - Reproduction interdite

- ⦿ Vrai
- ○ Faux

Même si Flask inclut Jinja, il s'agit de deux modules différents. Il est possible en théorie d'utiliser Jinja sans utiliser de framework Web. Cependant, l'intérêt peut être relativement limité.

Question 5

L'utilisation d'un moteur de template augmente le risque en termes de cybersécurité.

- ⦿ Vrai
- ○ Faux

Oui, car les moteurs de template rendent possible le risque d'attaque par injection. Il est important de se prémunir de ces attaques en utilisant l'échappement (escape) ou le sand-boxing (bac à sable) par exemple.

Tous droits réservés à STUDI - Reproduction interdite

Tous droits réservés à STUDI - Reproduction interdite